

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-211БВ-24

Студент: Отмахов Н.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.10.25

Москва, 2025

Постановка задачи

Вариант 5.

Отсортировать массив целых чисел при помощи четно-нечетной сортировки Бетчера

Общий метод и алгоритм решения

Использованные системные вызовы:

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);` – Создает поток с заданными атрибутами, который начинает выполнение функции `start_routine`
- `int pthread_join(pthread_t thread, void **retval);` – ожидает завершения указанного потока.
- `int pthread_mutex_lock(pthread_mutex_t *mutex);` – блокирует мьютекс. Предотвращает состояние гонки при одновременном доступе из нескольких потоков
- `int pthread_mutex_unlock(pthread_mutex_t *mutex);` – разблокирует мьютекс.\

Я реализовал программу, которая использует четно-нечетную сортировку Бэтчера для сортировки массива целых чисел. На вход программа получает число, которое является максимальным количеством потоков, а также два имени файла. В первом находятся числа, а во второй производится вывод отсортированного массива.

Алгоритм сортировки рекурсивный, и при каждом вызове, он разделяет входной массив на две части и сортирует каждую из них по отдельности. Для сортировки каждой половины создается отдельный поток, если количество активных потоков не превышает максимальное число. В противном случае вычисления выполняются последовательно

Также я использую `mutex`, чтобы предотвратить гонку.

Код программы

main.c

```
#include <fcntl.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "include/readfile.h"
#include <time.h>
#include "include/batcher_sort.h"

int max_threads;
int active_threads = 1;
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

int main (int argc, char *argv[])
{
    if (argc != 4)
```

```

{
    const char msg[] = "Using: ./program <num_of_threads> <input_file> <output_file>\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_SUCCESS);
}
const int32_t num_of_threads = atoi(argv[1]);
if (num_of_threads < 1)
{
    const char msg[] = "ERROR: Invalid num of threads\nNum of threads must be possitive integer\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int32_t input_fd = open(argv[2], O_RDONLY);
if (input_fd == -1)
{
    const char msg[] = "ERROR: Invalid filename\nCan't open input file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
int32_t output_fd = open(argv[3], O_WRONLY | O_CREAT);
if (output_fd == -1)
{
    const char msg[] = "ERROR: Invalid filename\nCan't open output file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    close(input_fd);
    exit(EXIT_FAILURE);
}

uint32_t error_status;

char *file_data = read_from_file(input_fd, &error_status);
switch (error_status)
{
    case 1:
    {
        const char msg[] = "ERROR: Memory Error\nCan't get memory to read file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        close(input_fd);
        close(output_fd);
        exit(EXIT_FAILURE);
    }
    break;
    case 2:
    {
        const char msg[] = "ERROR: Descriptor Error\nDescriptor can't read from input file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        close(input_fd);
        close(output_fd);
        exit(EXIT_FAILURE);
    }
    break;
}

```

```

}
close(input_fd);

uint32_t nums_size = 0;
int32_t *nums = convert_to_int(file_data, &error_status, &nums_size);

switch (error_status)
{
    case 1:
    {
        const char msg[] = "ERROR: Memory Error\nCan't get memory to convert data\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        close(output_fd);
        free(file_data);
        file_data = NULL;
        exit(EXIT_FAILURE);
    }
    break;
    case 2:
    {
        const char msg[] = "ERROR: Ptr Error\nFunction got wrong ptr\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        close(output_fd);
        free(file_data);
        file_data = NULL;
        exit(EXIT_FAILURE);
    }
    break;
}

free(file_data);

max_threads = num_of_threads;

clock_t start_tick, end_tick;
double elapsed_time;
start_tick = clock();

batcher_sort(nums, nums_size, 1);

end_tick = clock();

elapsed_time = (double)(end_tick - start_tick) / CLOCKS_PER_SEC * 1000;

{
    char msg[50];
    snprintf(msg, 50, "Time: %.3f ms\nMax threads: %d\n", elapsed_time, max_threads);
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}

error_status = write_nums_to_file(output_fd, nums, nums_size);

```

```

if (error_status == 1)
{
    const char msg[] = "ERROR: Descriptor ERROR\nDescriptor can't write to output file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    free(nums);
    nums = NULL;
    exit(EXIT_FAILURE);

}
free(nums);
nums = NULL;

return 0;
}

```

batcher_sort.c

```

#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include "../include/batcher_sort.h"

typedef struct {
    int32_t *arr;
    int32_t n;
    int32_t ascending;
} task_t;

void *sort_worker(void *arg) {
    task_t *t = (task_t *)arg;
    batcher_sort(t->arr, t->n, t->ascending);
    free(t);

    pthread_mutex_lock(&mtx);
    active_threads--;
    pthread_mutex_unlock(&mtx);

    return NULL;
}

void merge(int32_t *arr, int32_t n, int32_t ascending) {
    if (n <= 1) return;

    int32_t mid = n / 2;
    int32_t *left = malloc(mid * sizeof(int32_t));
    int32_t *right = malloc((n - mid) * sizeof(int32_t));

    memcpy(left, arr, mid * sizeof(int32_t));
    memcpy(right, arr + mid, (n - mid) * sizeof(int32_t));
}

```

```

int32_t i = 0, j = 0, k = 0;

while (i < mid && j < n - mid) {
    if ((ascending && left[i] <= right[j]) ||
        (!ascending && left[i] >= right[j])) {
        arr[k++] = left[i++];
    } else {
        arr[k++] = right[j++];
    }
}

while (i < mid) arr[k++] = left[i++];
while (j < n - mid) arr[k++] = right[j++];

free(left);
free(right);
}

void batcher_sort(int32_t *arr, int32_t n, int32_t ascending) {
    if (n <= 1)
    {
        return;
    }
    int32_t mid = n / 2;
    pthread_t t1 = 0, t2 = 0;
    int32_t t1_created = 0, t2_created = 0;

    pthread_mutex_lock(&mtx);
    if (active_threads < max_threads) {
        active_threads++;
        pthread_mutex_unlock(&mtx);

        task_t *args = malloc(sizeof(task_t));
        args->arr = arr;
        args->n = mid;
        args->ascending = ascending;

        if (pthread_create(&t1, NULL, sort_worker, args) == 0) {
            t1_created = 1;
        } else {
            pthread_mutex_lock(&mtx);
            active_threads--;
            pthread_mutex_unlock(&mtx);
            free(args);
            batcher_sort(arr, mid, ascending);
        }
    } else {
        pthread_mutex_unlock(&mtx);
        batcher_sort(arr, mid, ascending);
    }

    pthread_mutex_lock(&mtx);

```

```

if (active_threads < max_threads) {
    active_threads++;
    pthread_mutex_unlock(&mtx);

    task_t *args = malloc(sizeof(task_t));
    args->arr = arr + mid;
    args->n = n - mid;
    args->ascending = ascending;

    if (pthread_create(&t2, NULL, sort_worker, args) == 0) {
        t2_created = 1;
    } else {
        pthread_mutex_lock(&mtx);
        active_threads--;
        pthread_mutex_unlock(&mtx);
        free(args);
        batcher_sort(arr + mid, n - mid, ascending);
    }
} else {
    pthread_mutex_unlock(&mtx);
    batcher_sort(arr + mid, n - mid, ascending);
}

if (t1_created)
{
    pthread_join(t1, NULL);
}
if (t2_created)
{
    pthread_join(t2, NULL);
}

merge(arr, n, ascending);
}

```

Протокол работы программы

➤ ./program 1 test/test_32768.txt test/output.txt

Time: 7.295 ms

Max threads: 1

➤ ./program 2 test/test_32768.txt test/output.txt

Time: 13.227 ms

Max threads: 2

➤ ./program 8 test/test_32768.txt test/output.txt

Time: 25.394 ms

Max threads: 8

➤ ./program 12 test/test_32768.txt test/output.txt

Time: 67.608 ms

Max threads: 12

➤ ./program 16 test/test_32768.txt test/output.txt

Time: 81.982 ms

Max threads: 16

➤ ./program 1024 test/test_32768.txt test/output.txt

Time: 2837.814 ms

Max threads: 1024

➤ ./program 8096 test/test_32768.txt test/output.txt

Time: 5065.434 ms

Max threads: 8096

➤ ./program 16192 test/test_32768.txt test/output.txt

Time: 10090.617 ms

Max threads: 16192

Вывод

В ходе данной лабораторной работы я научился работать с потоками. Разобрался с различными проблемами и нюансами при работе с ними (гонка). Также получил следующие выводы, которые я изображу в виде таблицы

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	7.295	1.00	1.00
2	13.227	0.55	0.28
8	25.394	0.29	0.036
12	67.608	0.108	0.009
16	81.982	0.009	0.0056
1024	2837.814	0.0026	0.000003
8096	5065.434	0.0014	0.0000002
16192	10090.617	0.0007	0.000000045

Расчеты:

- Ускорение: $T1/Tn$, где $T1$ – время выполнения с 1 потоком,
 Tn – время выполнения с n потоками
- Эффективность: $Ускорение/n$

Анализ результатов:

По результатам мы видим, что наиболее эффективной программа оказалась, когда работала в одном потоке. Это связано с проблемами в реализации. Такие потери в эффективности связаны с многократным открытием и закрытием потоков в ходе рекурсивного вызова. Помимо этого мы можем заметить, что потери эффективности растут еще стремительнее, когда число потоков превышает число логических ядер процессора. В итоге вместо ускорения программы мы видим ее замедление. Это можно изменить, если изменить подход к реализации параллельной сортировки. Например, создать pool threads: изначально создать определенное количество потоков и передавать в них новые данные. Это сократит расходы на открытие и закрытие потоков.